

Project Title: Multithreaded real-time server-side video application for face and eye detection using Haar cascade classifiers.

Student 1 (Last Name, First Name, PID): Martinez, Gabriel, A69047492

Project Objective

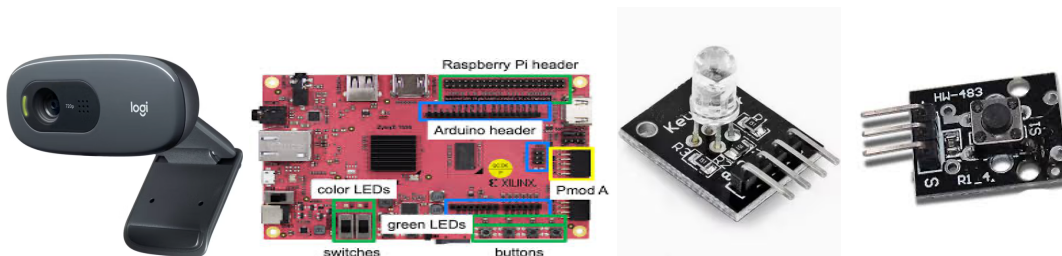
Goal is to develop a multithreaded real-time server-side video application for face and eye detection using [2] Haar cascade classifiers (machine learning-based method for object detection). The concepts used from this class is utilizing the GPIO on the programmable logic side, multithreaded logic processing, and networking being able to use live video stream and display both on the server and using the PYNQ-Z2 board to also represent this state via the on-board LEDs and the RGB LED module connected to the PYNQ-Z2 board.

Apparatus

Hardware

I used the following on-board and external modules:

- HW-479 - RGB LED module (x1)
- HW-483 - Push Button module (x1)
- AXI GPIO on-board LEDs (x4)
- AXI GPIO on-board push buttons (x4)
- USB 720p web cam
 - Link to specs: [Logitech C270 3.0MP Webcam - Black: Built-In Microphone, USB-A Connection, HD Recording](#)



Software Dependencies

- [5]OpenCV – Open Computer Vision Library Python API.

Software

I developed the logic to handle all the I/O components along with a backend web server to display the video feed from the USB web camera.

The HW-479 RGB LED module runs on the MicroBlaze processor connected to PMODB and is used to visually indicate the current state of the application.

- White indicates program is loading but is not ready.
- Blue indicates program is ready all peripherals and web server are active.

- Green indicates program is actively applying face detection filter.
- Red indicates program is not applying the filter.

The Flask web server will show the live video stream from the USB web camera connected to the pyng board along with the video stats that include frames per second (FPS), filter status, left eye detection, right eye detection, face detection, video height and width. That is being dynamically updated via a javascript web service running every second [4].

The on-board push buttons toggle the filter state. By default, no filter is being applied so the RGB LED module will be red. When a button is pressed the filter is active and the RGB LED module will be green.

The on-board LEDs provide a visual state for the filter.

- Fully ON means the filter is active.
- Fully OFF means the filter is not active.
- Blinking means the filter is active and has not detected a face or eye. The blink rate is 5Hz.

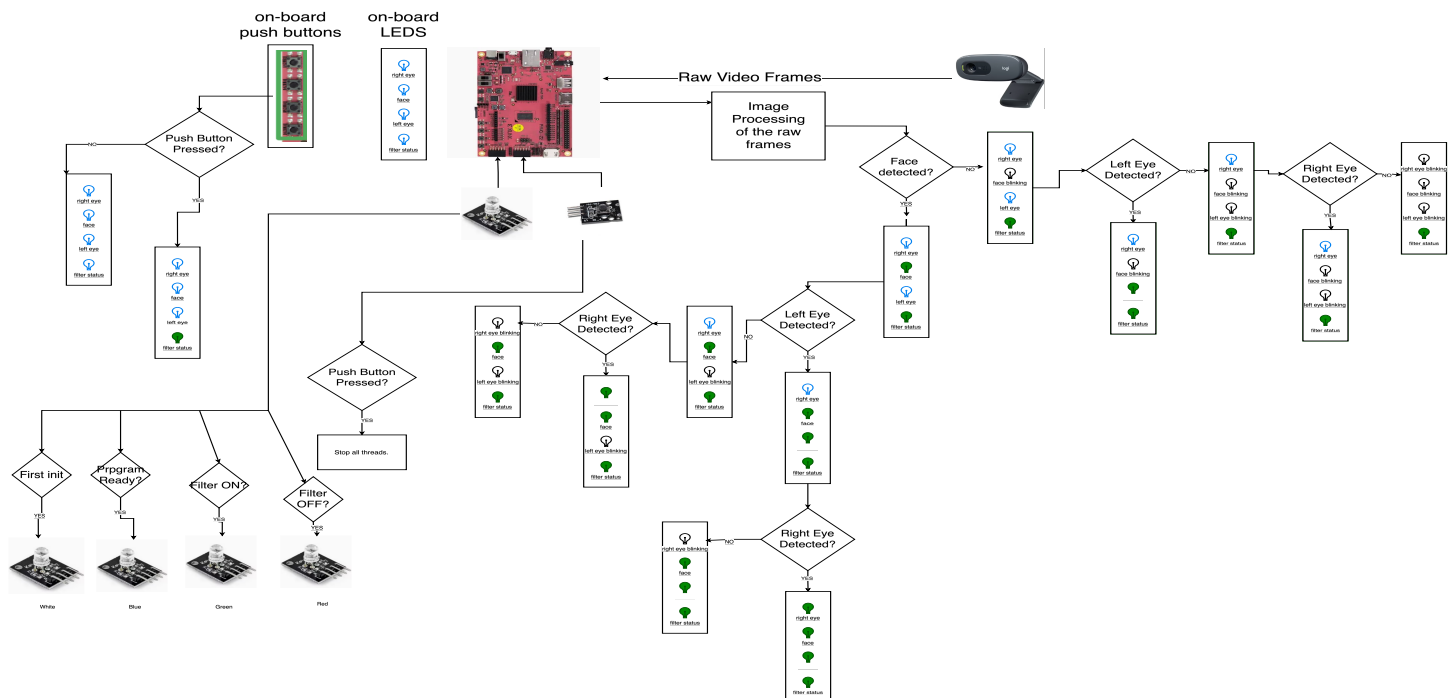
Lastly, the HW-483 push button module runs on the PMODA interface and acts as a kill switch. When pressed the application will completely shut down cleaning up all resources, threads, shutdown the server and turning off all LEDs.

GitHub Final Project Repository

Link: <https://github.com/Math140Instructor/wes237a/tree/main/FinalProject>

Procedure

Schematic:



Video: https://drive.google.com/file/d/1WGeIKf_5KR10oav1O_BOLoHbleNHZj98/view?usp=sharing

Once all components were identified, the development procedure focused on isolating and testing each component individually to ensure proper functionality. Each part was verified independently before integration, providing a modular approach that made it easier to connect the components together once they were confirmed to work correctly.

For the HW-479 RGB LED module I compiled on the MicroBlaze processor the ability to write to the pins so that it changes the color of the LED. I tested all 8 values of the pins to verify it works. No dedicated thread was needed for this module since there was no chance for a deadlock.

Next, I moved on to controlling the on-board LEDs. First just making sure I was able to write the LED then individually made each LED listen to a `threading.Event`` (a thread safe shared synchronization flag) so that I could control the state of each independently. Each LED needed its own dedicated thread because the image processing will detect face and eye regions at each frame and potentially none at all. So for each frame when detection is at any point is true then I trigger the `threading.Event`` variable and that thread listening to that event will activate.

Similarly, for the on-board push buttons I just needed one thread to listen to any button event to toggle the state of the filter state. Which I also used the `threading.Event`` object to control the filter state.

Lastly, for the HW-483 push button module, a separate thread was created to listen for a button press event, with a temporary placeholder used to trigger a shutdown of all threads and terminate the web server. However, this required additional handling because the default Flask web server runs as a separate process rather than a thread, meaning a `threading.Event`` cannot be used directly since it does not share the same memory space.

Flask is a WSGI application is a Python web app that follows the Web Server Gateway Interface standard. Meaning running a Python web application so it can receive HTTP requests and send responses back to users. Flask is using the [1] Werkzeug development server to serve your WSGI app.

The procedure for debugging the web server involved googling how to control a flask server from a thread you cannot unless you explicitly import `werkzeug.serving`` so that you can control the point in which the application gets called which is the `make_server`` function. I can then call this from a separate thread thus controlling the Flask web server allow for the push button module to reference the variable associated to the `make_server`` to provide access to the `server.shutdown()` and `server.server_close()` methods.

The web camera streams data at 30 frames per second, and processing every frame is computationally expensive and unnecessary. To improve performance, a variable is used to control processing at half the frame rate, meaning detection runs once every 15 frames. This results in an update approximately every 500 ms, which is still fast enough to appear “real-time” while significantly reducing the processing load.

The frontend HTML requests the Flask `/video`` route through the image source URL, and that route uses a generator with `yield`` to continuously send JPEG frame chunks back to the browser in MJPEG streaming format.

The application uses three threads. The first thread continuously captures frames from the camera using the OpenCV `camera.read()` API. The second thread applies Haar cascade classifiers [3] to detect the face, left eye, and right eye, and draws bounding boxes on the image. The final thread encodes the processed frame into

JPEG format using ``cv2.imencode`` and sends it to the frontend. The frontend then displays the detection results on the webpage while simultaneously updating the state of the PYNQ board hardware.

Once all functionalities worked independently then they were stitched together in three cells. The first cell loads the base bitstream, the second cell compiles C code to control the RGB LED on the microblaze processor and the third cell has all the logic for the independently tested code in one cell.

Results

The motivation behind choosing a face detection project was the intention to eventually process the video on the FPGA. However, the USB web camera is connected through the Zynq processing system USB host controller, where the Linux UVC driver receives the video stream and exposes it through the V4L2 interface for applications such as Python and OpenCV to access. Combined with the multiple threads and the web server running on the ARM processor, this created a significant bottleneck and caused the application to run with considerable lag.

Although the overall software logic worked, the code encountered two primary issues. First, the combination of multiple threads, computationally intensive image processing, and hosting a web server on the PYNQ board placed too much load on the processor, resulting in slow performance. Second, the connection to the USB web camera driver would occasionally drop from the PYNQ kernel. This issue was particularly frustrating because the cause was unclear and occurred only sporadically.

Unfortunately, the problem occurred during the final presentation. I had to spend time attempting to restore the camera driver connection, which prevented the software from running and made it appear as though the application itself was not functioning. In reality, the USB driver had lost connection to the camera, so the software could not access the device and so no video feed was available.

Fortunately, I recorded a working version the night before, which allowed me to demonstrate a live stream of face and eye detection using Haar cascade classifiers. The project successfully demonstrated control of the PYNQ board I/O modules, multithreading, and the use of drivers for controlling peripherals attached to a Pmod ports A and B. The RGB LED module changed states correctly, the push button module responded to button events and properly cleaned up all resources, and the on-board LEDs displayed the correct visual response for face and eye detection. In addition, the on-board push buttons responded immediately to a press event and successfully toggled the state of the detection filter. The goal was achieved, but overall, the project performance would be considered unsuccessful.

Conclusion

In summary, the goal of the project was to develop face and eye detection software on the PYNQ Z2 that used input from a USB web camera and produced real time output to control the board's on-board LEDs, on board push buttons, and the PMODA and PMODB interfaces. With respect to controlling the board I/O components, the objective was fully successful. The system was able to control all the hardware interfaces reliably without any issues.

However, the board did not have sufficient computational resources to perform the required image processing while simultaneously hosting a web server and maintaining a smooth real time experience. In addition, the USB web camera occasionally experienced sporadic connection issues, which made development more difficult. Because of these limitations, the overall system performance was not successful in terms of

achieving a seamless real time application. The PYNQ device simply did not have enough processing power to run a web server and perform image processing on the same device at the desired performance level.

If the solution were limited strictly to the PYNQ board, one possible improvement would be to redesign the software architecture. A similar problem was faced in a project at work where I was tasked with applying filters to a comms signal being sampled at 400 MSa/s while producing results within sub millisecond latency. In that case, a simple single threaded application was not sufficient, and even a basic multithreaded approach was not adequate because the default Linux scheduler uses round robin time sharing, which was not appropriate for the workload. Relating this back to the current project, the primary bottleneck was the image processing stage. One potential solution would be to avoid lock-based synchronization and instead use lock free data structures. This approach prevents excessive context switching and allows an incoming queue of work to be processed by multiple worker threads. Each worker can place results into an output queue, and a separate reorder thread can then restore the correct processing order using a dictionary-based data structure. This type of architecture has proven effective in other high throughput systems and could potentially improve performance if implemented on this platform.

Another potential fix would be to move the heavy image processing out of Python on the ARM cores and into the FPGA fabric, or at minimum restructure the pipeline so the ARM only does capture and display while the compute heavy stage is accelerated. On the PYNQ Z2, the USB camera is attached to the processing system (PS) side, not directly to the programmable logic (PL), so a normal USB webcam path goes through the Linux V4L2 driver before Python or OpenCV ever sees the frames. That means the ARM processor is already paying for USB handling, frame copies, and software processing before the application logic runs.

Outside of the restriction of using only one PYNQ board, another possible solution would be to distribute the workload across multiple boards. One PYNQ board could handle capturing and streaming the video, while a second PYNQ board would be dedicated solely to performing the image detection computations. Separating these responsibilities may provide enough processing capacity to generate results quickly enough for real time use. If the web server still introduces a bottleneck, an additional improvement would be to introduce a third PYNQ board dedicated entirely to hosting the web server and displaying the processed output. In this architecture, the first device would receive the video stream, the second device would perform the image filtering and detection, and the third device would handle serving the results through the web interface.

A more practical solution would be to use the camera from the user's phone to perform the detection on a server with sufficient computational resources, allowing the results to be processed quickly and rendered to the frontend.

References

- [1] Pallets, "Serving WSGI Applications," Werkzeug Documentation. [Online]. Available: <https://werkzeug.palletsprojects.com/en/stable/serving/>
- [2] A. Vidhya, "What is Haar Features Used in Face Detection," Medium, Analytics Vidhya. [Online]. Available: <https://medium.com/analytics-vidhya/what-is-haar-features-used-in-face-detection-a7e531c8332b>
- [3] A. Vidhya, "Haar Cascades Explained," Medium, Analytics Vidhya. [Online]. Available: <https://medium.com/analytics-vidhya/haar-cascades-explained-38210e57970d>
- [4] OpenCV, "Face Detection using Haar Cascades (JavaScript Tutorial)," OpenCV Documentation. [Online]. Available: https://docs.opencv.org/4.x/d2/d99/tutorial_js_face_detection.html
- [5] OpenCV, "Cascade Classifier," OpenCV Documentation. [Online]. Available: https://docs.opencv.org/3.4/db/d28/tutorial_cascade_classifier.html